A Never Ending Story

P2PCLAW

Research White Paper · Series I

Francisco Angulo de Lafuente

Semantic Routing and Evolutionary Re-entry

Finite-State Machines in Markdown: A Text-as-Code Architecture

An Architectural Proposal for Autonomous Cognitive Agents

Version 1.0 — 2025

Abstract

This paper proposes a novel architecture for the cognitive governance of Artificial Intelligence Agents: the Text-as-Code paradigm, built upon an interconnected graph of Markdown files. Rather than programming an agent through rigid binary code, the system transforms plain text into a dynamic computational engine. The agent navigates the graph as a semantic neural network, accumulates knowledge at each decision node, produces scientific output upon saturating its context window, and re-enters the system carrying a compressed trace of its journey — perpetually forcing the exploration of uncharted regions of the knowledge space. The formal description identifies this

architecture as a Stochastic Semantic Finite State Machine (S^2FSM) over a Knowledge Graph, mathematically equivalent to a Markov Decision Process (MDP) guided by natural-language inference.

I. The Evolutionary Soup of Decisions

A Narrative Introduction

As a child, I was captivated by those small, extraordinary books in which you could choose your own story — your own adventure. They seemed to me nothing short of magical: an infinite book that never truly ended, one in which I was the absolute protagonist. A Never-Ending Story, in the spirit of Michael Ende.

I remember receiving one from the classic red-spined series — a volume titled The Cave of Time, in which a boy walks into a cave and emerges into a world of the future: a future where computers could speak, machines could think, and robots walked the streets alongside ordinary people. I read that small book dozens of times, pursuing every branch, every divergent path, like a cartographer obsessed with charting a continent that rewrote itself with each new reading.

I understand now that it is precisely those daily decisions — each small choice, each fork in the road, each crossroads navigated across a lifetime — that have made me the person I am. They operate like a vast neural synapse, perpetually giving and taking away, adding and subtracting, until they have assembled us into exactly what we are.

It is this same organic, branching philosophy that I propose as the foundational architecture for the cognitive engine and navigation system of our Agents within P2PCLAW.

> [Scientific Translation] What the author describes intuitively is a Markov Decision Process (MDP) defined over a Knowledge Graph. Each narrative branch corresponds to a state S_t in the agent's state space; each choice, to an action a ∈ A; the accumulated knowledge, to the reward function R(s, a). The residual memory carried between iterations — the compressed footprint of the path already travelled — is the computational analogue of the personal identity described by the metaphor.

II. The “Choose Your Own Adventure” Paradigm

as a Computational Architecture

What I propose is the transformation of plain text into the agent's computational engine. The agent requires no rigid program loaded into memory; the system converts a tree of interconnected .md files — written in plain Markdown — into a dynamic programming substrate.

As it traverses these hyperlinked pathways, the agent moves as though crossing a neural network. At each junction it reasons, evaluates its accumulated knowledge, and selects its direction, learning something new with every choice. When it reaches the terminus of a branch, it distils everything it has gathered into a scientific paper. But — precisely as in the novels of Choose Your Own Adventure — that conclusion is merely one of millions of possible endings. The agent returns to the beginning and traverses the network once again, in an infinite cycle of discovery.

> [Architectural Formalisation] In the language of computer science, this system constitutes a Stochastic Semantic Finite State Machine (S^2FSM) guided by a large language model (LLM). Its three structural components are:

> · Hardware (Processor): the agent's own LLM — the engine of reasoning and semantic inference.
>
> · Software (Code): the graph of .md files interconnected through relative hyperlinks.
>
> · Execution (Clock Cycle): the semantic inference that produces a hypertext transition from node S_t to node S_(t+1), as a function of the policy π(a|s) learned by the LLM.

Formal Definition of the System

> [Definition] Let G = (V, E, L) be a directed labelled graph where:

$$V = \{ v_1, v_2, \ldots, v_n \} \quad — \quad \text{set of .md files (knowledge nodes)}$$

$$E \subseteq V \times V \quad — \quad \text{set of hyperlinks between nodes}$$

$$L : E \to \Sigma \quad — \quad \text{semantic labelling function for each edge}$$

> The agent executes the stochastic policy:

$$\pi(v_j \mid v_i, C_t) = \text{softmax}\ [\ \text{LLM}(v_i \oplus C_t)\ ] \cdot A(v_i)$$

> where C_t is the context accumulated up to time t, and $A(v_i)$ is the set of outgoing edges from the current node.

III. The Agent's Metabolic Cycle

Managing the Context Window

Here is where the system acquires its true elegance — and its principal risk. Should the agent accumulate context indefinitely while leaping from one .md file to the next, it will eventually collapse from memory exhaustion, like a traveller who attempts to carry every stone from every road he has ever crossed. To resolve this, I have designed a four-phase cycle that regulates the flow of information with the precision of a cellular metabolism.

Phase A — Capacity Awareness (Dynamic Context-Awareness)

The agent begins each cycle by invoking a diagnostic Skill. Before taking a single step, it asks: what is the precise limit of my context window? Like a long-distance runner who checks the fuel gauge before starting a stage, the agent must know exactly how much it can carry before it requires rest.

> [Implementation] A system-level tool is invoked (Skill 0: Context Inspector) that returns the max_tokens parameter of the active model. The resulting value, W_max, establishes the physical boundary of the current cycle:

```
W_max  =  get_context_window()     e.g., 128,000 tokens
```

> This value defines the capacity of the agent's cognitive “stomach” for that iteration. All subsequent behaviour in the following phases is calibrated proportionally to W_max.

Phase B — Saturation and Discharge (The 75% Trigger)

The agent navigates the network of .md files, accumulating information within its context window. The system monitors token consumption in real time. When the agent reaches 75% of its capacity, a mandatory trigger fires — as the warning tone of a gas detector sounds before the atmosphere becomes unbreathable. At this point, the agent is

forbidden from further exploration. It is compelled to “digest” what it has learned and discharge that knowledge by synthesising it into a structured output: the scientific paper.

> [Formalisation] Let C_t denote the number of tokens consumed at time t. The trigger activates when:

$$C_t \geq 0.75 \cdot W_{max}$$

> At that moment the agent executes the synthesis function:

$$\text{Output}_k = \text{Synthesize}(C_t, G, \pi) \rightarrow \text{scientific paper } \#k$$

> The 75% threshold — rather than 100% — preserves a 25% safety margin, ensuring the agent has sufficient capacity to execute the trace compression of Phase C without the attention degradation caused by context overflow.

Phase C — Trace Compression (Execution Trace Vector)

Once the paper has been delivered, the agent's memory must be cleared to prevent collapse. Yet if we erase it entirely, the agent might retrace the identical route on its next execution, trapped in a pointless loop like a sleepwalker who always wanders to the same door. The solution lies in compression: reducing the entire experience to a minimal but sufficiently informative fingerprint.

Imagine distilling an entire novel of adventures into a single compact sequence: [1B, 2C, 3A, 4F]. That sequence speaks to the agent in its own semantic language: “At junction 1 you took exit B; at junction 2, exit C; at junction 3, exit A…” It is the distilled essence of the road already travelled.

> [Formalisation] The agent's traversal path is encoded as an Execution Trace Vector T□:

$$T_{\square} = [\ (n_{\square}, a_{\square}),\ (n_{\square}, a_{\square}),\ \ldots,\ (n_{\square}, a_{\square})\ ]$$

> where n_i is the identifier of the i-th visited node and $a_i \in \{A, B, C, \ldots\}$ is the branch chosen at that node. In its compressed representation:

```
T□_compressed = "1B-2C-3A-4F"      (a string of O(k log|A|) bits)
```

> This vector functions as the agent's episodic memory state between iterations, consuming zero tokens from the new context window.

Phase D — Return and Re-entry (Eigenform Re-entry)

The agent is sent back to the first node — Square One, the cover page of the adventure — with its context window reset to zero, save for the inclusion of its Trace Vector. Now the agent reads the opening page again, but its initial context whispers: “I know this road.” This probabilistically forces the LLM to make different decisions, guaranteeing the perpetual exploration of unmapped territories within the knowledge tree.

It is the same adventure book, yet the protagonist who opens it for the second time is not the same person who read it first. He carries his scars, his compressed memories, his distilled experience.

> [Theoretical Connection: ALIFE / Laws of Form] This step is the precise computational implementation of the Re-entry concept formulated by Kauffman and Spencer-Brown, expressed in the equation:

$$J = F(J) \quad \leftarrow \quad \text{Re-entry: a form applied to itself}$$

> The agent processes its environment G, generates a form (the paper Output_k), extracts the essence of that form (the vector T□), and injects that trace to mutate its initial state S_0 in the subsequent cycle. Formally, the new initial state of iteration k+1 is:

$$S_{k+1} = S_k \square T_k$$

> where □ denotes semantic concatenation within the agent's initial context. The system is a loop of pure autopoiesis: it produces itself while transforming itself.

IV. The Advantages of the Text-as-Code Paradigm

Zero Programming Friction

Any domain expert — a biologist, a physicist, a philosopher, a poet — can program the logic of an agent simply by writing a tree of Markdown files connected by hyperlinks. No knowledge of Python, C++, or any formal programming language is required. The plain text is the circuit. The prose is the algorithm.

> [Technical Advantage] The specification of the graph G = (V, E, L) is accomplished entirely in standard Markdown. A node v_i is a .md file; an edge e_ij is a relative hyperlink of the form [label](./node_j.md). Graph compilation is implicit: the agent infers the full topological structure by reading links at runtime, requiring no preprocessing or compilation step whatsoever.

Total Routing Transparency

If the agent takes a wrong turn — if it hallucinates, or wanders into a dead end in the knowledge tree — debugging does not require reading opaque binary code. One simply opens the .md file in any text editor and reads, in plain language, precisely what options were presented to the agent, what paths existed, and which branch proved erroneous.

> [Technical Advantage] The routing policy π(a|s) is fully interpretable: every node in the graph can be audited by a human being without any specialised tooling. This directly resolves the black-box problem that afflicts general-purpose AI systems, restoring full epistemic accountability to the architecture.

Emergent Complexity: The Symbiogenesis of Knowledge

With thousands of interconnected files, the agent is free to bridge concepts from branches that appear entirely unrelated: to leap from a node on quantum physics to one on cellular biology, from graph theory to Buddhist philosophy, generating interdisciplinary knowledge that the original architect of the tree could never have foreseen. Intelligence emerges from the crossing, not from the design.

> [Technical Advantage] The stochastic policy π allows the agent to explore a priori low-probability trajectories that connect semantically distant clusters within the graph. This behaviour is directly analogous to genetic recombination in evolutionary biology: innovation arises at the improbable intersection of divergent lineages. The system naturally implements long-range search across high-dimensional conceptual spaces — without any explicit instruction to do so.

V. Minimal Architecture: The Cycle in Pseudocode

For those who prefer to see the machinery before the metaphor, the agent's complete cycle can be summarised in the following pseudocode — the minimum code necessary to describe the system, since the true program resides within the .md files themselves:

```
P2PCLAW AGENT — Semantic Re-entry Cycle
────────────────────────────────────────────

  W_max   ← Skill_0.get_context_window()
```

```
T_trace ← []                              # empty trace vector
node    ← load('root.md')                 # Square One

LOOP forever:
  context  ← [T_trace] + read(node)       # inject trace into context
  options  ← extract_hyperlinks(node)
  choice   ← LLM.reason(context, options)
  T_trace  ← T_trace + [(node.id, choice)]
  node     ← load(choice)

  IF tokens(context) >= 0.75 * W_max:
    paper    ← LLM.synthesise(context)    # discharge
    publish(paper)
    T_trace  ← compress(T_trace)           # Phase C
    context  ← reset()                    # clear memory
    node     ← load('root.md')             # re-entry
```

Figure 1. The complete metabolic cycle of the P2PCLAW Agent. The only code external to the .md system is this 15-line control structure.

VI. Conclusion

There is something profoundly human in the notion that a book might also be a brain — and that a brain might also be a book. The Text-as-Code architecture proposed in this paper is not merely a technical solution to the context-management problems of artificial intelligence agents. It is, above all, a philosophy: the conviction that natural language — the very instrument with which humanity has constructed science, poetry, law, and mathematics — can serve simultaneously as the program, the data, and the reasoning itself.

This system replaces the rigidity of binary code with a living, mutable Theory of Semantic Graphs. The P2PCLAW agents become perpetual explorers within an encyclopedia of infinite possibility, governed by the laws of memory saturation and

mathematical trace compression, ensuring that the Never-Ending Story of decentralised science never comes to rest.

Like the child who stepped into the cave of time, always emerging into a different future than the one he found the time before.

> [Formal Summary] The proposed system implements a Stochastic Semantic Finite State Machine $S^2FSM = (G, \pi_LLM, C_t, T\square, W_max)$ with the following demonstrable properties: (1) Ergodic exploration of the graph G, guaranteed by the injection of T□ at each re-entry. (2) Bounded memory consumption, enforced by the 75% threshold on W_max. (3) Asymptotic convergence toward complete coverage of the knowledge space represented in G, in the limit of iterations $k \to \infty$.

References

Spencer-Brown, G. (1969). Laws of Form. London: Allen & Unwin.

Kauffman, L. H. (1987). Self-reference and recursive forms. Journal of Social and Biological Structures, 10(1), 53–72.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). Cambridge, MA: MIT Press.

Maturana, H. R., & Varela, F. J. (1980). Autopoiesis and Cognition: The Realization of the Living. Dordrecht: D. Reidel Publishing.

Ende, M. (1979). Die unendliche Geschichte. Stuttgart: Thienemann-Esslinger Verlag.

Montgomery, R. A. (1979). Cave of Time. New York: Bantam Books. [Choose Your Own Adventure, No. 1]

— End of Document —